

# Laboratorio 4 - Collezioni, persistenza e trasformazioni

## Obiettivo

Evolgere il programma del Laboratorio 3 in un **motore di analisi con persistenza**, capace di caricare i dati in memoria, indicizzarli, modificarli dinamicamente tramite trasformazioni e salvarli.

Il laboratorio serve a consolidare:

- **Collezioni e persistenza:** utilizzo di `BTreeMap` per indicizzare e ordinare i dati, e del framework `Serde` per la serializzazione/deserializzazione in JSON (con `serde-json`)
- **Ownership e borrowing:** utilizzo di riferimenti mutabili (`&mut`) per applicare trasformazioni sul posto ai dati e di riferimenti immutabili (`&`) per il raggruppamento e l'analisi, approfondendo la gestione del ciclo di vita
- **Iteratori e chiusure:** utilizzo di catene di iteratori (`filter`, `fold`, `for_each`) e l'applicazione di trasformazioni ai dati passate sotto forma di chiusure

## Come iniziare: GitHub Classroom

Questo laboratorio utilizza **GitHub Classroom** per la distribuzione e l'autovalutazione.

## Istruzioni

1. Accettate l'assignment tramite il link fornito dal docente su GitHub Classroom
2. Clonate il repository creato automaticamente sul vostro computer: `git clone <url-del-vostro-repository>`
3. All'interno del repository troverete:
  - Un file `Cargo.toml` a cui dovrete aggiungere le necessarie librerie con `cargo add` (es. `clap`, `anyhow`, `serde` con feature `derive`, `serde_json`)
  - Una cartella `src/` con il file `main.rs` come punto di ingresso
  - Una cartella `tests/` contenente i test di integrazione
  - Due file nella cartella `samples/`, `ordini.csv` e `studenti.csv` con dati di esempio
4. Per compilare il progetto: `cargo build`
  - Per eseguire il programma: `cargo run -- <argomenti>`
5. Per eseguire i test forniti: `cargo test`

**Importante:** la cartella `tests/` non va modificata in alcun modo, ma solo visionata per comprendere cosa ci si aspetta dal programma. I test forniti verificano la correttezza della vostra implementazione

## Workflow consigliato

- Fate commit frequenti e con messaggi significativi
- Eseguite `cargo test` regolarmente per verificare i progressi
- Scrivete i vostri **test unitari** mano a mano che implementate nuove funzioni
- Fate push sul repository remoto per salvare il vostro lavoro:

Shell

```
git add <file-modificati>
git commit -m "feat: aggiunto support al group-by"
# o, se fosse stato un bugfix:
# git commit -m "fix: corretta gestione di filtri multipli"
git push
```

## Specifiche funzionali

### Retrocompatibilità

Tutte le modalità di aggregazione del Lab 3 (`count`, `sum`, `avg`, `min`, `max`) continuano a funzionare con la stessa interfaccia e lo stesso formato di output base. L'operazione di filtraggio (`--filter`) mantiene la sintassi del Lab 3 ma viene estesa per ammettere **più occorrenze** sulla stessa riga di comando (vedi sezione dedicata)

### Interfaccia

Il programma supporta ora l'esecuzione tramite la seguente CLI:

```
cargo run -- <file_input> [opzioni]
```

Il `<file_input>` può avere estensione `.csv` oppure `.json`.

### Argomenti nuovi o modificati

| Argomento                             | Obbligatorio | Descrizione                                                                                                                                 |
|---------------------------------------|--------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <code>--filter &lt;expr&gt;</code>    | No           | Espressione di filtraggio (stessa sintassi del Lab 3). <b>Può essere specificata più volte</b> : le condizioni sono combinate in AND logico |
| <code>--transform &lt;expr&gt;</code> | No           | Applica una trasformazione a una colonna nel formato <code>colonna=operazione</code>                                                        |

| Argomento                              | Obbligatorio                                                                          | Descrizione                                                                                                                  |
|----------------------------------------|---------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <code>--group-by &lt;col&gt;</code>    | No                                                                                    | Nome della colonna per cui raggruppare i risultati prima dell'aggregazione                                                   |
| <code>--export &lt;filename&gt;</code> | No, ma mutualmente esclusivo rispetto a <code>--mode</code> o <code>--group-by</code> | Esporta i dati attualmente elaborati in un file con il formato determinato dall'estensione indicata (supportate: JSON e CSV) |

## Comportamento atteso

Il programma opera secondo la seguente catena di elaborazione:

1. **Caricamento:** Lettura da file `.csv` o `.json` e popolazione di un contenitore in memoria (es. `Vec<Row>` all'interno di una struttura `Dataset`)
2. **Trasformazione (`--transform`):** Se presente l'argomento, il programma itera su tutte le righe accedendovi in modo mutabile (via `&mut self`) e applicando l'operazione richiesta tramite chiusure
3. **Esportazione (`--export`):** Se l'utente ha richiesto l'esportazione in JSON/CSV, i dati (eventualmente modificati) vengono salvati su disco e il programma termina la sua esecuzione
4. **Indicizzazione (`--group-by`):** Se richiesto, il dataset viene scandito per popolare un indice (`BTreeMap`) raggruppando i riferimenti immutabili (`&Row`) in base al valore assunto dalla colonna specificata come chiave
5. **Filtraggio e aggregazione:** Per ogni gruppo precedentemente individuato (o sull'intero dataset se `--group-by` non è presente), vengono applicati in AND tutti i `--filter` forniti dall'utente e successivamente eseguita l'aggregazione indicata da `--mode` (con l'eventuale colonna bersaglio `--column`)

## Filtri multipli

L'argomento `--filter` può essere fornito **più volte** sulla stessa invocazione. In quel caso le condizioni devono essere combinate in **AND logico**: una riga supera il filtraggio solo se soddisfa tutte le espressioni indicate.

Esempio:

```
cargo run -- libretti.csv --mode avg --filter voto>18 --filter
"corso=Programmazione Di Sistema" --column voto
```

Ciascuna espressione testuale deve essere creata in una chiusura indipendente del tipo `Box<dyn Fn(&Row) -> bool>`; l'insieme delle chiusure va raccolto in un `Vec<Box<dyn Fn(&Row) -> bool>>` e applicato alla pipeline con un singolo passaggio. Ogni chiusura

deve possedere i valori estratti dall'espressione originale (indice di colonna, operatore, valore di confronto), così da sopravvivere alla funzione che la crea.

Le eventuali righe di output di configurazione relative ai filtri devono riportare **una riga filter: ... per ciascuna espressione fornita**, nell'ordine in cui appaiono sulla riga di comando.

## Trasformazione dati

L'argomento `--transform` accetta stringhe del tipo `nome_colonna=operazione`. Le operazioni valide, che agiscono esclusivamente su campi di tipo **Testo**, sono:

- `lowercase`: Converte il testo in minuscolo.
- `uppercase`: Converte il testo in maiuscolo.

Queste operazioni devono essere implementate utilizzando **chiusure** o puntatori a funzione che alterano i dati *sul posto* (senza crearne copie).

## Raggruppamento (group-by)

Se specificato `--group-by <colonna>`, il programma crea un raggruppamento per ciascun valore distinto assunto da `<colonna>`. L'output deve elencare i gruppi elaborati **in ordine alfabetico** basandosi sul nome della chiave.

I gruppi per cui **nessuna riga** supera i filtri non devono essere stampati.

Esempi:

Dato il file `studenti.csv` fornito nel repository Github, ed eseguendo il seguente comando:

```
cargo run -- ./samples/studenti.csv --group-by corso --mode avg --column voto --filter voto>18 --filter "corso=Programmazione Di Sistema"
```

Ci si attende il seguente output:

```
Shell
mode: avg
column: voto
group_by: corso
filter: voto>18
filter: corso=Programmazione Di Sistema
Programmazione Di Sistema: 24.6
rows_analyzed: 5
```

Invece, eseguendo: `cargo run -- ordini.csv --group-by regione --mode sum --column importo --filter "prodotto=laptop" --filter importo>1000` sul seguente file, una semplificazione di `ordini.csv`:

```
None
cliente, regione, prodotto, importo
Rossi, nord, laptop, 1200
Bianchi, sud, tastiera, 45
Verdi, nord, laptop, 980
Neri, centro, monitor, 300
Gialli, nord, monitor, 280
Blu, sud, laptop, 1500
```

Ci si attende il seguente output:

```
Shell
mode: sum
column: importo
group_by: regione
filter: prodotto=laptop
filter: importo>1000
nord: 1200.0
sud: 1500.0
rows_analyzed: 2
```

Notare come il gruppo **centro** non compaia nell'output: nessuna sua riga supera i filtri, quindi il gruppo viene omesso. Il dataset di prova fornito è solo un campione e la vostra implementazione deve funzionare su qualsiasi CSV o JSON conforme allo schema descritto.

## Esportazione e importazione JSON

Il file prodotto deve rispettare esattamente la seguente struttura JSON (i campi in azzurro sono esempi, la lunghezza delle liste è arbitraria, ma devono essere rispettati i vincoli indicati):

```
JSON
{
  "headers": ["nome", "corso", "voto"], // lista di nomi
  "column_types": ["Text", "Text", "Integer"], // lista di tipi
  // deve avere la stessa lunghezza di headers
  "rows": [
    // Liste di valori: devono avere la stessa lunghezza di
    // headers e avere tipi coerenti con column_types
    ["Alice", "PDS", 28],
    ["Bob", "PDS", 25]
  ]
}
```

Se viene richiesto un export in JSON o CSV (es. `--export data.json` o `--export data.csv`), il programma si limita a serializzare il contenuto del dataset in memoria (post-trasformazione) nel formato indicato.

L'estensione determina anche il comportamento in lettura: se l'input fornito come primo argomento è un `.json`, il programma non tenta di leggerlo come CSV ma sfrutta `serde` per la deserializzazione, procedendo poi ad applicare liberamente `--transform`, `--filter` o `--group-by`.

## Architettura

### Ownership e borrowing

A differenza del Lab 3, dove il dato era consumato al momento, ora i dati devono essere conservati in memoria all'interno di una struttura `Dataset` che contiene un `Vec<Row>`.

- La trasformazione (`--transform`) avviene tramite accesso mutabile al dataset (`&mut self`), iterando sulle righe e applicando una chiusura che modifica i campi in-place.
- Il raggruppamento (`--group-by`) avviene tramite riferimenti immutabili alle righe (`&Row`), costruendo una `BTreeMap<String, Vec<&Row>>` che indicizza le righe senza duplicarle. Questo richiede una corretta gestione dei lifetime.

### Iteratori e chiusure

Dovete implementare i passaggi della pipeline sfruttando l'uso degli iteratori, alcuni consigli:

- L'aggiornamento *sul posto* del dataset dovrà avvenire iterando sulle righe (`&mut Row`) e applicando la logica proveniente dalla chiusura definita dalla opzione `--transform`
- L'implementazione di `group-by` si presta bene all'uso dell'operatore `.fold()`
- Cercate, quanto possibile, di sostituire blocchi `for` di logica complessa con espressioni funzionali concatenate

## Test unitari (requisito obbligatorio)

A differenza dei laboratori precedenti, in questo laboratorio dovete **scrivere voi** i test unitari per le componenti principali del vostro codice.

### Requisiti minimi

- Almeno **2 test** dedicati al `group-by`: verificate che un dataset definito da voi in un vettore si ramifichi correttamente in mappe distinte in base ai valori attesi

- Almeno **2 test** sulla Serializzazione / Deserializzazione JSON: verificate il cosiddetto meccanismo di *round-trip* assicurandovi che serializzando e deserializzando dei dati essi siano uguali in ingresso ed in uscita
- Almeno **1 test** sulla trasformazione: forzate un aggiornamento del dataset basato su una chiusura e assicuratevi che i valori siano effettivamente mutati nel contenitore originale
- Almeno **1 test** sulla composizione di filtri multipli: verificate che due o più chiusure costruite tramite la factory, combinate in AND, selezionino esclusivamente le righe che soddisfano *tutte* le condizioni

I test unitari vanno scritti all'interno dei moduli sorgente, usando la convenzione standard di Rust con `#[cfg(test)]` e `#[test]`.

## Struttura del progetto

Struttura minima consigliata (evoluzione del Lab 3):

- **main.rs** - Inizializzazione CLI, orchestrazione della sequenza di lettura, trasformazione, indice, filtro e output.
- **cli.rs** - Definizione migliorata degli argomenti attesi (`clap`).
- **csv.rs** - Parsing testuale base (ripreso ed esteso) per ingestione del CSV.
- **dataset.rs** (o simili) - Astrazione del contenitore in-memory (`Dataset` con `Vec<Row>`), metodi per trasformazione in-place, esportazione e costruzione indici tramite iteratori, e derive di Serde per il formato JSON. Qui risiederanno gran parte dei nuovi test unitari.
- **aggregator.rs** e **analysis.rs** - Ripresi dal Lab 3 per le logiche dei filtri e degli aggregatori matematici finali.

## Gestione errori (requisito obbligatorio)

Il programma **non deve andare in panic** in uso normale. Affidatevi alla gestione tramite `anyhow::Result` e messaggi descrittivi via terminale su `stderr` (`eprintln!`).

Devono essere gestiti (in aggiunta a quelli del Lab 3):

- `--transform` senza uguale, con operazione errata o mirato a una colonna inesistente o incompatibile (es. un intero non è un testo da fare `uppercase`).
- `--group-by` indirizzato ad una colonna inesistente
- `--export` specificato in parallelo ad altri comandi incompatibili
- Errata formattazione o file corrotto in fase di lettura JSON da disco
- Una o più espressioni `--filter` malformate o riferite a colonne inesistenti: la validazione deve avvenire al momento della compilazione di tutte le chiusure, **prima** di iniziare la scansione del dataset

# Criteri di autovalutazione

## 35% - Correttezza funzionale

- L'export JSON è formattato correttamente
- Il parser riesce a ingerire e ricostruire il file JSON senza perdite rispetto all'equivalente input CSV
- Le trasformazioni applicate da riga di comando alterano con successo i dati in memoria
- Raggruppamento corretto e retrocompatibilità con le modalità storiche (`count`, `sum`, `avg`, `min`, `max`)
- Tutti i test forniti (`cargo test --test integration`) passano

## 25% - Architettura, iteratori e lifetime

- Gestione corretta del possesso e del borrowing: trasformazioni tramite `&mut self`, raggruppamento tramite riferimenti immutabili con lifetime espliciti
- Implementazione delle elaborazioni basata fluidamente su catene di combinatori per Iteratori
- Le chiusure per i passaggi filtro e trasformazione catturano efficacemente l'ambiente, solo quando strettamente necessario; i filtri multipli sono raccolti in un `Vec<Box<dyn Fn(&Row) -> bool>>` e applicati tramite una singola catena di iteratori
- `BTreeMap` utilizzata efficacemente per gli indici di ricerca su base stringa ordinata

## 20% - Robustezza

- Nessun panic accidentale
- Messaggi di errore dettagliati sui fallimenti, rendendo all'utente facile l'individuazione di eventuali problemi nel file
- Trattamento adeguato di eccezioni legate alla deserializzazione del JSON/CSV o di errori di I/O (in fase di export)
- Exit code di processo corretti

## 10% - Test unitari

- Presenza di almeno i 6 test richiesti integrati nei vari file sorgenti
- Significatività dei test (i test falliscono se la logica introdotta è deliberatamente fallata)

## 10% - Processo di sviluppo

- Repository Git con messaggi significativi, visibili con `git log`
- Il progetto compila interamente senza generare *warnings*